

KARTIST

Profesyonel Dijital Kart Tasarım Platformu

⚡ FİNAL ÖDEVİ — SPRINT RAPORU

Sprint 1 & Sprint 2 Teknik Geliştirme Raporu

Bu rapor, Kartist web uygulamasının Sprint 1 (Güvenlik & Profiller) ve Sprint 2 (UI Kişiselleştirme & Canlı Destek) kapsamında yapılan tüm teknik geliştirmeleri detaylı olarak açıklamaktadır.

📅 Mart 2026

🔗 ASP.NET Core 9

🗄️ MSSQL + Dapper

Sprint 1: Güvenlik ve Profiller

Hafta 1–2

1.1 SecurityHeaders Middleware (CSP, XSS, Clickjacking)

1.2 InputValidator — SQL Injection & XSS Koruması

1.3 Brute Force Koruması & Hesap Kilitleme

1.4 İki Faktörlü Kimlik Doğrulama (2FA)

1.5 Giriş Loglama Sistemi

1.6 Gelişmiş Profil Dashboard Tasarımı

Sprint 2: UI Kişiselleştirme & Destek

Hafta 3–4

2.1 Editör Arayüz Kişiselleştirme (Tema & Yerleşim)

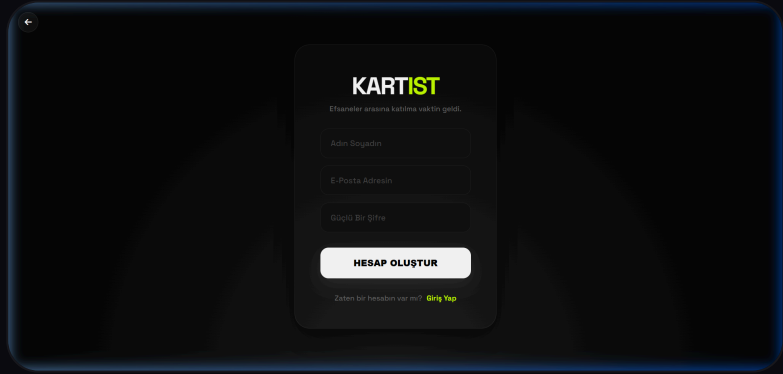
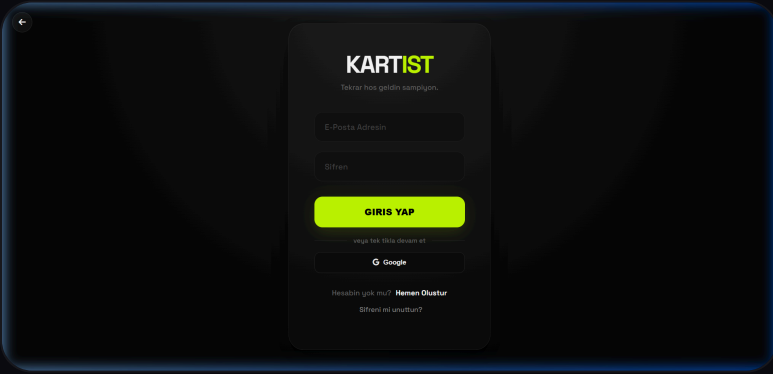
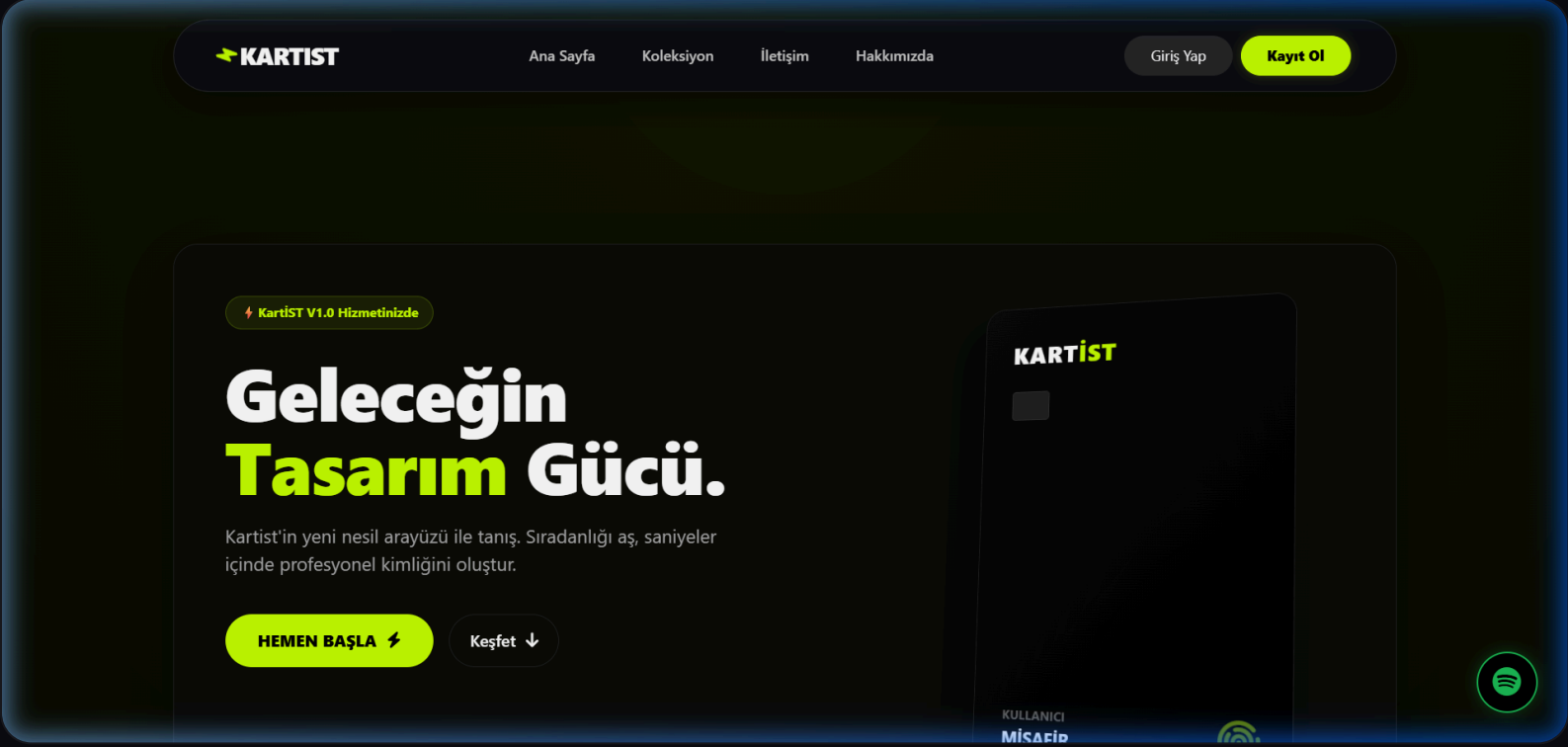
2.2 SignalR Canlı Destek (Live Chat)

Teknik Mimari & Teknoloji Yığını

Veritabanı Şeması

Uygulama Arayüz Görselleri

Kartist platformunun ana sayfa, giriş, kayıt ve tasarım editörü ekranları.





HAZIR ŞABLONLAR

Yükleniyor...

KATMANLAR

Destek



Güvenlik ve Profiller

Sistemin kale gibi sağlamlaştırılması ve kullanıcı deneyiminin kişiselleştirilmesi.

Hafta 1-2 </> 6 Özellik 5+ Dosya Değişikliği

SİBER GÜVENLİK

SecurityHeaders Middleware

Tüm HTTP yanıtlarına güvenlik başlıkları ekleyen özel bir ASP.NET Core middleware geliştirildi. Bu katman, uygulamayı XSS, Clickjacking, MIME Sniffing ve diğer yaygın web saldırılarına karşı korur.

Middleware/SecurityHeadersMiddleware.cs

```
// Her HTTP yanıtına eklenen güvenlik başlıkları
context.Response.Headers.Append("X-XSS-Protection", "1; mode=block");
context.Response.Headers.Append("X-Frame-Options", "SAMEORIGIN");
context.Response.Headers.Append("X-Content-Type-Options", "nosniff");
context.Response.Headers.Append("Referrer-Policy", "strict-origin-when-cross-origin");

// Content Security Policy – hangi kaynakların yükleneceğini kontrol eder
string csp = "default-src 'self'; script-src 'self' ...";
context.Response.Headers.Append("Content-Security-Policy", csp);

// Sunucu bilgisi ifşasını engelle
context.Response.Headers.Remove("Server");
context.Response.Headers.Remove("X-Powered-By");
```

- **X-XSS-Protection:** Tarayıcının yerleşik XSS filtrelerini aktif eder
- **X-Frame-Options:** Clickjacking saldırılarını engeller, sadece aynı origin iframe'e izin verir
- **Content-Security-Policy:** Script, style, font, image, connect gibi kaynakların whitelist'ini belirler
- **Permissions-Policy:** Geolocation, kamera, mikrofon gibi API'lere erişimi kapatır
- **Sunucu Bilgisi Gizleme:** Server ve X-Powered-By header'ları kaldırılarak teknoloji ifşası engellenir
- **Performans:** Statik dosyalar (CSS, JS, img) için header ekleme atlanarak performans optimize edildi

🔍 InputValidator — Girdi Doğrulama Sistemi

Kullanıcı girdilerini SQL Injection ve XSS saldırılarına karşı filtreleyen merkezi bir doğrulama sınıfı oluşturuldu. Tüm controller'larda kullanıcı girdisi alınan her noktada bu validator kullanılıyor.

📁 Helpers/InputValidator.cs

```
public static string SanitizeHtml(string input) {  
    // <script>, <iframe>, onclick= gibi tehlikeli içerikleri temizler  
    input = Regex.Replace(input, @"<script[^\>]*>[\s\S]*?</script>", "");  
    input = Regex.Replace(input, @"javascript\s*:", "");  
    return input.Trim();  
}  
  
public static bool IsValidInput(string input) {  
    // SQL anahtar kelimelerini (UNION, SELECT, DROP vb.) kontrol eder  
    string[] dangerousChars = { "'", "--", "union", "select", "drop", ... };  
    // ...  
}
```

METOD	İŞLEVİ
SanitizeHtml()	HTML etiketlerini, script bloklarını ve event handler'ları temizler
IsValidInput()	SQL Injection anahtar kelimelerini tespit eder
IsValidPrompt()	AI prompt'larında XSS pattern'lerini kontrol eder
IsValidEmail()	E-posta formatını RFC standardına göre doğrular
IsValidImageFile()	Yüklenen dosya uzantısını whitelist'e göre kontrol eder
SanitizeForDisplay()	HTML encode yaparak güvenli görüntüleme sağlar

🔒 Brute Force Koruması & Hesap Kilitleme

Giriş sayfasında art arda başarısız giriş denemelerini tespit eden ve belirli bir eşik aşıldığında hesabı geçici olarak kilitleyen bir mekanizma geliştirildi.

📁 Controllers/AccountController.cs — Giriş Metodu

```
// Başarısız giriş sayacını artır
int yeniSayac = (user.BasarisizGirisSayisi ?? 0) + 1;

if (yeniSayac >= 5) { // 5 başarısız deneme sonrası kilitler
    db.Execute("UPDATE Kullanicilar SET BasarisizGirisSayisi = @s,
        HesapKilitliMi = 1, KilitBitisTarihi = @t WHERE Email = @e",
        new { s = yeniSayac, t = DateTime.Now.AddMinutes(15), e = email });
}

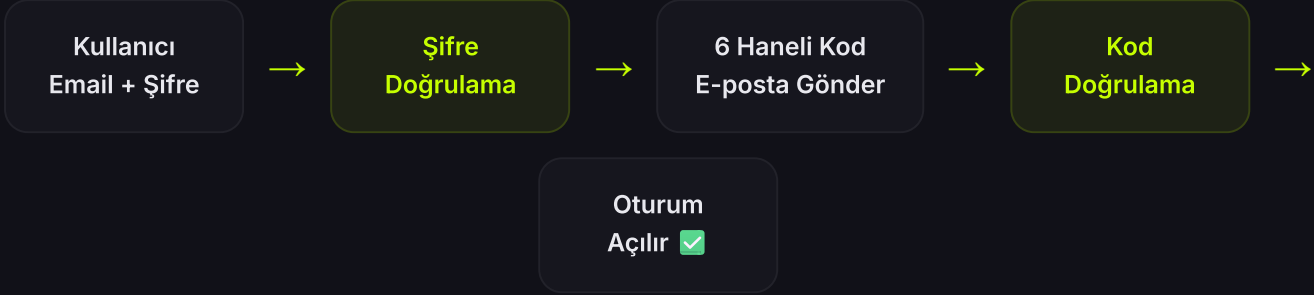
// Başarılı girişte sayacı sıfırla
db.Execute("UPDATE Kullanicilar SET BasarisizGirisSayisi = 0 WHERE Email = @e");
```

- **5 başarısız deneme** sonrası hesap **15 dakika** süreyle kilitlenir
- Başarılı giriş yapıldığında sayaç otomatik olarak **sıfırlanır**
- Kilit süresi dolduğunda hesap otomatik olarak **açılır**
- Veritabanında BasarisizGirisSayisi, HesapKilitliMi ve KilitBitisTarihi kolonları ile takip edilir

İki Faktörlü Kimlik Doğrulama (2FA)

E-posta tabanlı 2FA sistemi geliştirildi. Kullanıcılar profil ayarlarından 2FA'yı aktif ettiklerinde, her giriş denemesinde e-postalarına tek kullanımlık doğrulama kodu gönderilir.

2FA AKIŞ DİYAGRAMI



- **İkiFactorKodlari** tablosunda kodlar saklanır (Email, Kod, BitisTarihi, Kullanildi)
- Kodların **geçerlilik süresi** vardır, süresi dolan kodlar kabul edilmez
- **"Tekrar Gönder"** özelliği — spam engeli ile birlikte (60 sn bekleme)
- Kullanılan kodlar **Kullanildi = 1** olarak işaretlenir, tekrar kullanılamaz
- Profil sayfasından **tek tıkla açma/kapama** (toggle switch)
- Google ile giriş yapan kullanıcılar için de 2FA desteği

KARTIST

Tekrar hoş geldin sampiyon.

GİRİŞ YAP

veya tek tıkla devam et

G Google

Hesabın yok mu? **Hemen Oluştur**

Sifreni mi unuttun?

Giriş Loglama Sistemi

Her giriş denemesi (başarılı veya başarısız), IP adresi ve User-Agent bilgisiyle birlikte GirişLoglari tablosuna kaydedilir. Bu, şüpheli aktivitelerin tespitini sağlar.

Controllers/AccountController.cs

```
db.Execute(@"INSERT INTO GirişLoglari
(KullaniciEmail, IpAdresi, BasariliMi, UserAgent)
VALUES (@email, @ip, @basarili, @ua)",
new { email, ip = remoteIp, basarili = basariliMi, ua = userAgent });
```

KOLON	VERİ TİPİ	AÇIKLAMA
KullaniciEmail	NVARCHAR(256)	Giriş deneyen kullanıcının e-postası
IpAdresi	NVARCHAR(50)	İstemcinin IP adresi
BasariliMi	BIT	Giriş başarılı mı?
Tarih	DATETIME	İşlem zamanı (UTC)
UserAgent	NVARCHAR(500)	Tarayıcı/cihaz bilgisi

Gelişmiş Profil Dashboard Tasarımı

Kullanıcıların profillerini portfolyo gibi sergileyebileceği, modern ve premium hissiyatlı bir dashboard tasarlandı. Glassmorphism efektleri, animasyonlu istatistik kartları ve responsive tab yapısı kullanıldı.

- Hero Banner:** Radial gradient ve floating orb animasyonlarıyla dinamik karşılama alanı
- Profil Kartı:** Glassmorphism efektli, -70px margin ile hero üzerine taşan modern kart
- İstatistik Kartları:** Proje, Favori, Satış, Paylaşım sayılarını animasyonlu sayaçla gösteren 4'lü grid
- Tab Sistemi:** Projelerim / Favoriler / Ayarlar arasında geçiş yapan modern tab navigation
- Proje Kartları:** Hover'da scale efekti, overlay gradient, düzenle/kopyala/sil butonları
- Ayarlar Paneli:** Şifre değiştirme, biyografi düzenleme (canlı karakter sayacı), 2FA toggle
- Responsive:** 920px ve 600px breakpoint'lerinde otomatik layout adaptasyonu

Tasarım Kararları

Dark tema (#0a0a0f arka plan) üzerine neon yeşil (#c6ff00) vurgu rengi kullanıldı. Inter font ailesi, subtle animasyonlar (slide-up, fade-in) ve hover micro-interaction'lar ile premium bir kullanıcı deneyimi hedeflendi. CSS değişkenleriyle (custom properties) tutarlı bir tasarım sistemi oluşturuldu.

UI Kişiselleştirme & Destek

Kullanıcıların kendi çalışma alanlarını yönetebilmesi ve anında destek alması.

📅 Hafta 3-4 </> 2 Ana Özellik 📁 3+ Dosya Değişikliği

ARAYÜZ KİŞİSELLEŞTİRME

🛠️ Editör Arayüz Kişiselleştirme

Tasarım editöründe kullanıcıların çalışma tarzına göre arayüzü özelleştirebilmesi sağlandı. Araç kutusu yerleşimi (sol/sağ) ve karanlık/neon tema seçenekleri localStorage ile kalıcı hale getirildi.

📁 Views/Home/Tasarim.cshtml — Editör Tercihleri

```
// Kullanıcı tercihlerini localStorage'dan yükle
function initEditorPrefs() {
  const theme = localStorage.getItem('editorTheme') || 'dark';
  const toolsSide = localStorage.getItem('toolsSide') || 'left';

  // Tema uygula
  document.body.classList.toggle('light-theme', theme === 'light');

  // Araç kutusu konumunu değiştir
  if (toolsSide === 'right') {
    document.querySelector('.main-body').classList.add('layout-tools-right');
  }
}
```

- **Araç Kutusu Yerleşimi:** Sol veya sağ tarafa taşıyabilme — layout-tools-right CSS sınıfıyla grid-template-areas değişimi
- **Tema Desteği:** Karanlık (varsayılan) ve aydınlık tema seçenekleri
- **localStorage Kalıcılığı:** Tercihler tarayıcıda saklanır, her girişte hatırlanır
- **Responsive Editör:** Masaüstü, tablet ve mobil cihazlarda adaptive layout
- **Ayarlar Sekmesi:** Sidebar'daki "Ayar" tabından tüm tercihleri yönetme

💡 Teknik Detay

Editör, **Fabric.js** kütüphanesiyle canvas manipülasyonu yapıyor. Canvas üzerinde metin ekleme, şekil çizme, resim yükleme, QR kod oluşturma, katman yönetimi, undo/redo ve buluta kaydetme gibi gelişmiş özellikler mevcut. Responsive tasarım için 480px, 768px, 992px breakpoint'lerinde farklı layout'lar uygulanıyor.



HAZIR ŞABLONLAR

Yükleniyor...



KATMANLAR

Destek



SignalR Canlı Destek Sistemi (Live Chat)

Kullanıcıların tasarım yaparken sistem yöneticilerine anında ulaşabileceği, gerçek zamanlı bir canlı sohbet sistemi SignalR hub yapısıyla geliştirildi.

SIGNALR MİMARİSİ



Hubs/AdminHub.cs

```
public class AdminHub : Hub
{
    // Kullanıcı mesajı → Adminlere ilet
    public async Task SendUserMessage(string email, string message) {
        var payload = new { from = "user", email, message = cleanMessage };
        await Clients.Group("admins").SendAsync("SupportMessage", payload);
        await Clients.Group(email).SendAsync("SupportMessage", payload);
    }

    // Admin yanıtı → Kullanıcıya ilet
    public async Task SendAdminMessage(string email, string message) {
        var payload = new { from = "admin", email, message = cleanMessage };
        await Clients.Group("admins").SendAsync("SupportMessage", payload);
        await Clients.Group(email).SendAsync("SupportMessage", payload);
    }
}
```

- **Gerçek Zamanlı İletişim:** WebSocket tabanlı anında mesajlaşma, sayfa yenileme gerektirmez
- **Grup Bazlı Yönlendirme:** Her kullanıcı kendi email grubuna, adminler "admins" grubuna katılır
- **Çift Yönlü:** SendUserMessage ve SendAdminMessage metotlarıyla iki yönlü iletişim
- **Oturum Doğrulama:** OnConnectedAsync'te admin oturumu kontrol edilerek rol bazlı erişim sağlanır
- **Spam Koruması:** İstemci tarafında duplicate mesaj tespiti — 1.5 saniye içinde aynı mesajı engeller
- **UI Entegrasyonu:** Editör sayfasında sağ altta Spotify benzeri destek widget'ı, badge ile okunmamış mesaj göstergesi

Program.cs — SignalR Kaydı

```
builder.Services.AddSignalR(); // DI Container'a SignalR ekle
app.MapHub<AdminHub>("/adminHub"); // Hub endpoint'ini tanımla
```


Teknoloji Yığını & Mimari

Projede kullanılan teknolojiler ve mimari kararlar.

KATMAN	TEKNOLOJİ	SÜRÜM / NOT
Backend Framework	ASP.NET Core MVC	.NET 9
ORM / Data Access	Dapper (Micro-ORM)	Parametrized queries ile SQL Injection koruması
Veritabanı	Microsoft SQL Server	MSSQL — Dapper ile bağlantı
Gerçek Zamanlı	SignalR	WebSocket tabanlı canlı iletişim
Kimlik Doğrulama	Cookie Authentication + Google OAuth	2FA desteğiyle
Frontend	Vanilla JS + Fabric.js	Canvas editörü için
CSS	Custom CSS + CSS Variables	Dark theme, glassmorphism, responsive
Güvenlik	Custom Middleware + InputValidator	CSP, XSS, CSRF koruması
CSRF Koruması	AntiForgeryToken	AutoValidateAntiforgeryToken global filtre
Sıkıştırma	Brotli + Gzip	Response compression middleware

Sprint 1-2 Kapsamında Eklenen Tablolar & Kolonlar

Program.cs içindeki EnsureDatabaseSchema metodu ile otomatik oluşturulan şema değişiklikleri.

Kullanıcılar Tablosu — Yeni Kolonlar

KOLON	TİP	SPRİNT	AÇIKLAMA
BasarisizGirisSayisi	INT	Sprint 1	Art arda başarısız giriş sayısı
HesapKilitliMi	BIT	Sprint 1	Hesap kilitli mi?
KilitBitisTarihi	DATETIME	Sprint 1	Kilidin otomatik açılacağı tarih
IkifactorAktif	BIT	Sprint 1	2FA aktif mi?
Biyografi	NVARCHAR(500)	Sprint 1	Kullanıcı bio metni
ProfilResmi	NVARCHAR(500)	Sprint 1	Profil fotoğrafı URL'i
UyelikBitisTarihi	DATETIME	Sprint 1	PRO üyelik bitiş tarihi

Yeni Tablolar

TABLO	SPRİNT	AÇIKLAMA
IkifactorKodlari	Sprint 1	2FA doğrulama kodları (Email, Kod, BitisTarihi, Kullanildi)
GirisLoglari	Sprint 1	Tüm giriş denemelerinin logu (Email, IP, Başarı, UserAgent, Tarih)

Sonuç

Sprint 1 ve Sprint 2 kapsamında Kartist platformuna **6 güvenlik özelliği** (Security Headers Middleware, InputValidator, Brute Force koruması, 2FA, Giriş loglama, CSRF), **gelişmiş profil dashboard'u**, **editör kişiselleştirme** ve **SignalR canlı destek** sistemi başarıyla entegre edilmiştir. Tüm özellikler, veritabanı şeması otomatik güncelleme (EnsureDatabaseSchema) mekanizmasıyla uyumlu çalışmakta ve responsive tasarımla mobil cihazlarda da sorunsuz şekilde kullanılabilir.

